

Complete set of 300 multiple-choice questions (MCQs) with answers and concise explanations

1) Core Python Programming (10)

1. In Python, what is the result of `type(3/2)`?

A) int; B) float; C) Decimal; D) Fraction

Answer: B

Explanation: / performs true division and returns float.

2. What does `print(2**3)` output?

A) 6; B) 8; C) 9; D) 5

Answer: B

Explanation: ** is exponentiation; $2^3 = 8$.

3. Which keyword defines a function?

A) def; B) func; C) define; D) lambda

Answer: A

Explanation: def declares named functions; lambda creates anonymous ones.

4. `is` compares:

A) values; B) types; C) identities; D) lengths

Answer: C

Explanation: `is` checks object identity, not equality.

5. Which is immutable?

A) list; B) dict; C) str; D) set

Answer: C

Explanation: Strings are immutable; lists/dicts/sets are mutable.

6. `len("Hello\n")` returns:

A) 5; B) 6; C) 7; D) 8

Answer: B

Explanation: Includes newline `\n` as one character.

7. `round(2.675, 2)` yields:

A) 2.67; B) 2.68; C) 2.66; D) 2.70

Answer: A

Explanation: Binary float rounding causes 2.675 to be represented slightly less than 2.675.

8. Valid variable name?

A) 1var; B) var-1; C) `_var1`; D) var 1

Answer: C

Explanation: Names cannot start with digits; hyphens/spaces invalid; underscore ok.

9. `print(bool([]))` outputs:

A) True; B) False; C) None; D) 0

Answer: B

Explanation: Empty sequences are falsy.

10. `range(5)` generates:

A) 0..5; B) 0..4; C) 1..5; D) 1..4

Answer: B

Explanation: Start at 0, stop is exclusive.

2) Data Structures: Lists, Tuples, Dictionaries, Sets (10)

1. Which creates a tuple?

A) (1,); B) [1,]; C) {1:2}; D) {1,}

Answer: A

Explanation: (1,) is a single-element tuple; {1,} is a set.

2. `list.append(x)` does:

A) add at front; B) add at end; C) insert at pos 0; D) sort list

Answer: B

Explanation: `append` appends to the tail.

3. Remove key `k` safely from dict `d`:

A) `del d[k]`; B) `d.pop(k, None)`; C) `d.remove(k)`; D) `d.discard(k)`

Answer: B

Explanation: `pop(k, None)` avoids `KeyError` if missing.

4. Sets are:

A) Ordered; B) Allow duplicates; C) Unordered unique items; D) Indexable

Answer: C

Explanation: Sets store unique elements without order.

5. `a=[1,2]`; `b=a`; `a.append(3)` → `b` becomes:

A) [1,2]; B) [1,2,3]; C) [3]; D) error

Answer: B

Explanation: `b` references same list object.

6. `d.items()` returns:

A) list of keys; B) list of values; C) view of (key,value); D) generator

Answer: C

Explanation: Returns a dynamic view of key-value pairs.

7. `x = {}`; `type(x)` is:

A) set; B) dict; C) tuple; D) list

Answer: B

Explanation: `{}` is an empty dict; empty set is `set()`.

8. `set.add(x)` average complexity:

A) $O(n)$; B) $O(\log n)$; C) $O(1)$; D) $O(n^2)$

Answer: C

Explanation: Hash-based insertion is amortized constant time.

9. `list.insert(i,x)` affects:

A) tail only; B) head only; C) shifts items right; D) shifts items left

Answer: C

Explanation: Elements at/after `i` shift right.

10. Merge dicts (Py3.9+):

A) `d1 + d2`; B) `d1 | d2`; C) `d1.append(d2)`; D) `dict(d1,d2)`

Answer: B

Explanation: `|` is the dict union operator.

3) List Comprehension, Iterators and Iterables (10)

1. `[x*x for x in range(3)]` is:

A) generator; B) list; C) set; D) tuple

Answer: B

Explanation: Square brackets produce a list.

2. `(x*x for x in range(3))` is:

A) list; B) generator; C) set; D) tuple

Answer: B

Explanation: Parentheses without brackets create a generator.

3. Iterable means:

A) has `__next__`; B) has `__iter__`; C) has `__len__`; D) is list

Answer: B

Explanation: Iterable must return an iterator via `__iter__`.

4. `next(iter([1,2]))` returns:

A) 1; B) 2; C) `StopIteration`; D) 0

Answer: A

Explanation: First element of iterator is 1.

5. Set comprehension syntax:

A) `{x for x in ...}`; B) `[x for x in ...]`; C) `(x for x in ...)`; D) `{x:...}`

Answer: A

Explanation: Braces produce a set comprehension.

6. Dict comprehension example:

A) `{x: x*x for x in range(3)}`; B) `{x for x in range(3)}`; C) `[x: x*x]`; D) `(x:x)`

Answer: A

Explanation: Key-value pairs for dictionaries.

7. `any([])` returns:

A) True; B) False; C) None; D) error

Answer: B

Explanation: any of empty iterable is False.

8. `all([True,0])` returns:

A) True; B) False; C) None; D) error

Answer: B

Explanation: 0 is falsy; all needs every item True.

9. `zip([1,2],[3,4])` length:

A) 4; B) 2; C) 3; D) 1

Answer: B

Explanation: Zips until shortest list length—here 2 pairs.

10. `enumerate(['a','b'],start=1)` first tuple:

A) (0,'a'); B) (1,'a'); C) (2,'a'); D) ('a',1)

Answer: B

Explanation: start=1 begins counting at 1.

4) Map and Filter (10)

1. `list(map(str, [1,2]))` →

A) ['1','2']; B) [1,2]; C) ('1','2'); D) error

Answer: A

Explanation: map applies str to each element.

2. `filter(None, [0,1,2,""])` yields:

A) [0,1,2,""]; B) [1,2]; C) [0,""]; D) []

Answer: B

Explanation: None keeps truthy elements only.

3. `map(len, ["a","ab"])` returns:

A) [1,2]; B) iterator; C) error; D) ['1','2']

Answer: B

Explanation: In Py3, map returns an iterator; listify to see [1,2].

4. `list(filter(lambda x:x%2==0, range(5)))` →

A) [0,2,4]; B) [1,3]; C) []; D) [2,4]

Answer: A

Explanation: Filters evens from 0 to 4.

5. map vs list comprehension:

A) map always faster; B) list comp often clearer; C) list comp returns iterator; D) map returns set

Answer: B

Explanation: Readability and Pythonic style favor comprehensions.

6. `list(map(lambda x:x+1, []))` →

A) []; B) [1]; C) error; D) None

Answer: A

Explanation: Mapping over an empty list returns empty list.

7. filter keeps elements where function returns:

A) False; B) True; C) None; D) 0

Answer: B

Explanation: Truthy results pass the filter.

8. list(map(int, ["1", "2a"])) →

A) [1,2]; B) error; C) [1, None]; D) ['1', '2a']

Answer: B

Explanation: int("2a") raises ValueError.

9. Compose evens then square using map/filter:

A) map then filter; B) filter then map; C) both same; D) none

Answer: B

Explanation: Filter first, then map on the reduced set.

10. list(filter(str.isalpha, "a1b!")) →

A) ['a', 'b']; B) ['a', '1']; C) []; D) ['!']

Answer: A

Explanation: Keeps alphabetic characters only.

5) Object Oriented Programming in Python (10)

1. Method's first parameter in instance methods is conventionally:

A) self; B) this; C) obj; D) cls

Answer: A

Explanation: self refers to the instance.

2. Class variable defined:

A) inside **init**; B) at class level; C) per instance; D) in method only

Answer: B

Explanation: Defined directly in class block.

3. @classmethod first arg is:

A) self; B) cls; C) this; D) klass

Answer: B

Explanation: Receives the class as the first parameter.

4. @staticmethod can access:

A) self; B) cls; C) neither by default; D) both

Answer: C

Explanation: No implicit self or cls is passed.

5. Constructor method:

A) **new**; B) **init**; C) **call**; D) **repr**

Answer: B

Explanation: `__init__` initializes the instance post-creation.

6. Inheritance syntax:

A) `class B[A];`; B) `class B(A);`; C) `class B->A;`; D) class B inherits A

Answer: B

Explanation: Parent class in parentheses.

7. Multiple inheritance:

A) No; B) Yes; C) Only via mixin; D) Only abstract

Answer: B

Explanation: Python supports multiple base classes.

8. `super()` is used to:

A) access globals; B) call parent methods; C) create singleton; D) freeze class

Answer: B

Explanation: Calls methods from the superclass in MRO.

9. Private attribute convention:

A) `_name`; B) `_name`; C) `name`; D) `NAME`

Answer: B

Explanation: Double leading underscores trigger name mangling.

10. Properties via `@property` enable:

A) read-only; B) computed attributes; C) validation; D) all

Answer: D

Explanation: Properties can customize getters/setters/deleters.

6) Exception Handling (10)

1. Catch exception syntax:

A) `try/handle`; B) `try/except`; C) `do/catch`; D) `try/catch`

Answer: B

Explanation: Python uses `try/except` blocks.

2. `finally` block runs:

A) only on success; B) always; C) only on exception; D) never

Answer: B

Explanation: `finally` executes regardless of exceptions.

3. Raise exception:

A) `throw`; B) `raise`; C) `except`; D) `error`

Answer: B

Explanation: `raise` triggers an exception.

4. Best-practice to catch broad exceptions:

A) `except;`; B) `except Exception;`; C) `except BaseException;`; D) `except Error:`

Answer: B

Explanation: Catch Exception to avoid SystemExit/KeyboardInterrupt.

5. Access exception object:

A) except E as e; B) catch(e); C) except as e; D) handle(e)

Answer: A

Explanation: except SomeError as e: binds the exception.

6. assert x>0 does:

A) silently pass; B) raises AssertionError if False; C) logs; D) returns bool

Answer: B

Explanation: Failing assertion raises AssertionError.

7. try/except/else else runs:

A) always; B) on no exception; C) on exception; D) never

Answer: B

Explanation: else runs if no exception in try.

8. Custom exception base:

A) inherits Exception; B) inherits BaseError; C) inherits Error; D) function

Answer: A

Explanation: User-defined exceptions should inherit Exception.

9. Multiple excepts order:

A) random; B) specific first; C) general first; D) alphabetical

Answer: B

Explanation: Specific handlers must precede general ones.

10. Resource cleanup commonly in:

A) try; B) except; C) finally; D) else

Answer: C

Explanation: finally ensures cleanup executes.

7) Python OOP Concepts (10)

1. Polymorphism refers to:

A) many forms; B) one form; C) overcomposition; D) compile-time typing

Answer: A

Explanation: Same interface, different implementations.

2. Encapsulation hides:

A) implementation; B) interface; C) data always; D) nothing

Answer: A

Explanation: Internal details are hidden from users.

3. Abstraction focuses on:

A) details; B) essentials; C) performance only; D) syntax

Answer: B

Explanation: Presents essential features, hides complex reality.

4. Method overloading in Python is:

A) built-in by signature; B) mimicked via defaults/*args; C) not possible; D) only via decorators

Answer: B

Explanation: Python does not have signature-based overloading.

5. Duck typing means:

A) type checks strict; B) behavior-based typing; C) only inheritance; D) static typing

Answer: B

Explanation: "If it quacks like a duck," focus on methods/behavior.

6. Interface-like patterns achieved via:

A) abstract base classes; B) enforced interfaces; C) macros; D) traits only

Answer: A

Explanation: abc module defines abstract bases.

7. Multiple dispatch provided by:

A) built-in; B) functools.singledispatch; C) typing.dispatch; D) none

Answer: B

Explanation: singledispatch dispatches on first arg type.

8. Mixin is:

A) concrete type; B) helper class giving methods; C) interface; D) module

Answer: B

Explanation: Mixins add reusable functionality across classes.

9. Liskov Substitution Principle means:

A) subclasses replace bases seamlessly; B) bases replace subclasses; C) static only; D) performance guarantee

Answer: A

Explanation: Derived types should be usable via base references.

10. @property setters use:

A) @set; B) @.setter; C) @setter; D) @mutate

Answer: B

Explanation: Decorator attaches a setter to the property.

8) Polymorphism and Operator Overloading (10)

1. Overload + via:

A) plus; B) add; C) sum; D) op

Answer: B

Explanation: __add__ defines behavior for +.

2. `==` overload method:
A) **equal**; B) **eq**; C) **cmp**; D) **compare**
Answer: B
Explanation: `__eq__` handles equality.
3. `len(obj)` calls:
A) **size**; B) **len**; C) **length**; D) **repr**
Answer: B
Explanation: `__len__` returns length as integer.
4. `str(obj)` calls:
A) **repr**; B) **str**; C) **format**; D) **doc**
Answer: B
Explanation: `__str__` for user-friendly printing.
5. Rich comparison `<` is:
A) **lt**; B) **gt**; C) **le**; D) **ge**
Answer: A
Explanation: `__lt__` implements "less than."
6. Callable objects implement:
A) **call**; B) **exec**; C) **run**; D) **do**
Answer: A
Explanation: `__call__` enables instance invocation like a function.
7. Iteration uses:
A) **iter** and **next**; B) **len**; C) **get**; D) **list**
Answer: A
Explanation: `__iter__` returns iterator; iterator defines `__next__`.
8. Context manager requires:
A) **enter** and **exit**; B) **start**; C) **open**; D) **with**
Answer: A
Explanation: Used by `with` statements.
9. Hashable objects define:
A) **hash** and **eq**; B) **len**; C) **repr**; D) **copy**
Answer: A
Explanation: Consistent hashing requires equality and hash.
10. Membership in uses:
A) **has**; B) **in**; C) **contains**; D) **member**
Answer: C
Explanation: Container membership is `__contains__`.
-

9) Magic Functions (Dunder Methods) (10)

1. Object representation in console typically uses:
A) **doc**; B) **repr**; C) **str**; D) **print**
Answer: B
Explanation: repr() is for unambiguous debugging display.
2. Attribute access fallback:
A) **getattr**; B) **getattribute**; C) **setattr**; D) **dir**
Answer: A
Explanation: Called only if attribute not found normally.
3. Intercept all attribute access:
A) **getattr**; B) **getattribute**; C) **setattr**; D) **slots**
Answer: B
Explanation: __getattribute__ runs for *every* access.
4. Prevent dynamic attributes via:
A) **dict**; B) **slots**; C) **hash**; D) **copy**
Answer: B
Explanation: __slots__ restricts attributes to predefined names.
5. Item access obj[key] uses:
A) **getitem**; B) **get**; C) **key**; D) **index**
Answer: A
Explanation: Indexing is implemented via __getitem__.
6. Slicing handled by:
A) **slice**; B) **getitem** with slice; C) **getslice**; D) **indexslice**
Answer: B
Explanation: Slices pass a slice object to __getitem__.
7. with obj: enters via:
A) **open**; B) **enter**; C) **start**; D) **with**
Answer: B
Explanation: Context manager protocol uses __enter__.
8. format(obj) calls:
A) **format**; B) **str**; C) **repr**; D) **doc**
Answer: A
Explanation: __format__ handles format() and f-strings with spec.
9. Numeric conversion int(obj) via:
A) **int**; B) **convert**; C) **num**; D) **cast**
Answer: A
Explanation: Implements integer conversion.
10. Boolean context if obj: uses:
A) **truth**; B) **bool**; C) **len** or **bool**; D) **true**

Answer: C

Explanation: If `__bool__` absent, `__len__` decides truthiness.

10) Abstract Method & Class, Empty Class, Data Class (10)

1. Abstract methods use decorator:

A) `@abstract`; B) `@abstractmethod`; C) `@interface`; D) `@abc`

Answer: B

Explanation: `abc.abstractmethod` marks abstract methods.

2. Abstract base class module:

A) `abc`; B) `base`; C) `abstract`; D) `core`

Answer: A

Explanation: The `abc` module provides ABCs.

3. Instantiate ABC with unimplemented abstract methods?

A) Yes; B) No; C) Only via metaclass; D) Only with **new**

Answer: B

Explanation: Python prevents instantiation until implemented.

4. Empty class syntax:

A) `class C(): pass`; B) `class C: empty`; C) `class C()`; D) `class C: {}`

Answer: A

Explanation: `pass` creates an empty class body.

5. Dataclass decorator:

A) `@dataclass`; B) `@data`; C) `@record`; D) `@struct`

Answer: A

Explanation: From `dataclasses` import `dataclass`.

6. Dataclass default `__repr__`:

A) not provided; B) auto-generated; C) error; D) minimal

Answer: B

Explanation: Shows field names and values.

7. Frozen dataclass:

A) mutable; B) immutable; C) no `init`; D) no `repr`

Answer: B

Explanation: `frozen=True` prevents attribute assignment.

8. Dataclass field defaults via:

A) `field(default=...)`; B) `default=...`; C) `@default`; D) `param`

Answer: A

Explanation: Use `dataclasses.field` for advanced defaults.

9. Post-init hook:

A) **post**; B) **post_init**; C) **after**; D) `__init__2`

Answer: B

Explanation: Runs after `__init__` to adjust derived fields.

10. Dataclass ordering enabled by:

A) `order=True`; B) `compare=True`; C) `sorted=True`; D) `lt=True`

Answer: A

Explanation: `order=True` adds comparison methods.

11) NumPy (10)

1. Import alias commonly:

A) `import numpy as np`; B) `import numpy as n`; C) `import np`; D) `from numpy import *`

Answer: A

Explanation: Community convention uses `np`.

2. Create array:

A) `np.array([1,2])`; B) `numpy([1,2])`; C) `array([1,2])`; D) `np.arr([1,2])`

Answer: A

Explanation: `np.array` constructs `ndarray`.

3. Shape of `np.zeros((2,3))`:

A) (3,2); B) (2,3); C) 6; D) (2,)

Answer: B

Explanation: Two rows, three columns.

4. Matrix multiplication operator:

A) `*`; B) `@`; C) `**`; D) `//`

Answer: B

Explanation: `@` performs matrix multiply.

5. Broadcasting allows:

A) incompatible shapes; B) aligning dimensions; C) slower ops; D) no vectorization

Answer: B

Explanation: Expands smaller shapes to match larger along axes.

6. `np.arange(3)` yields:

A) [1,2,3]; B) [0,1,2]; C) [0,2,4]; D) [3,2,1]

Answer: B

Explanation: Starts at 0, step 1, stop exclusive.

7. Random array:

A) `np.random.rand(2,2)`; B) `np.rand(2,2)`; C) `random(2,2)`; D) `np.random(2,2)`

Answer: A

Explanation: `rand` returns uniform [0,1) floats.

8. Boolean indexing:

A) `a[a>0]`; B) `a>0[a]`; C) `a.where`; D) `mask(a)`

Answer: A

Explanation: Use a boolean mask to filter elements.

9. Copy vs view: `a.view()` is:

A) deep copy; B) shallow view; C) no change; D) list

Answer: B

Explanation: Shares data buffer; different array object.

10. `np.where(cond, x, y)` returns:

A) indices; B) element-wise choose; C) bool; D) mask

Answer: B

Explanation: Picks elements from x or y based on condition.

12) Pandas (10)

1. Import alias commonly:

A) import pandas as pd; B) import pandas as ps; C) import pd; D) from pandas import *

Answer: A

Explanation: Community convention uses pd.

2. Create Series:

A) `pd.Series([1,2])`; B) `pd.array([1,2])`; C) `pd.list([1,2])`; D) `Series([1,2])`

Answer: A

Explanation: Series is 1D labeled array.

3. Read CSV:

A) `pd.read_csv('x.csv')`; B) `pd.csv('x.csv')`; C) `pd.read('x.csv')`; D) `pd.load_csv('x.csv')`

Answer: A

Explanation: `read_csv` loads tabular data.

4. Select by label:

A) `df.iloc`; B) `df.loc`; C) `df.at`; D) `df.iat`

Answer: B

Explanation: `loc` uses labels; `iloc` uses integer positions.

5. GroupBy sum:

A) `df.groupby('col').sum()`; B) `df.group('col').sum()`; C) `df.sum('col')`; D) `df.aggregate('sum')`

Answer: A

Explanation: Standard groupby aggregation.

6. Handle missing values:

A) `df.fillna(...)`; B) `df.na(...)`; C) `df.null(...)`; D) `df.replace_na`

Answer: A

Explanation: `fillna` fills NaNs.

7. Merge DataFrames:
A) `pd.join(df1,df2)`; B) `pd.merge(df1,df2)`; C) `df1.concat(df2)`; D) `df1.union(df2)`
Answer: B
Explanation: merge performs SQL-like joins.
8. Concatenate:
A) `pd.concat([df1,df2])`; B) `df1.concat(df2)`; C) `pd.merge`; D) `pd.stack`
Answer: A
Explanation: concat combines along an axis.
9. Convert dtype:
A) `df.astype(...)`; B) `df.dtype(...)`; C) `df.type(...)`; D) `df.cast(...)`
Answer: A
Explanation: astype changes column dtypes.
10. Describe stats:
A) `df.stats()`; B) `df.describe()`; C) `df.summary()`; D) `df.analyze()`
Answer: B
Explanation: describe gives summary stats.
-

13) Seaborn (10)

1. Typical import:
A) `import seaborn as sns`; B) `import seaborn as sb`; C) `import sns`; D) `from seaborn import *`
Answer: A
Explanation: sns is the convention.
2. Scatterplot:
A) `sns.barplot`; B) `sns.scatterplot`; C) `sns.lineplot`; D) `sns.pairplot`
Answer: B
Explanation: scatterplot plots x vs y points.
3. Heatmap:
A) `sns.heat`; B) `sns.heatmap`; C) `sns.mapheat`; D) `sns.map`
Answer: B
Explanation: Visualizes matrix-like data.
4. Pairwise plots:
A) `sns.pairplot`; B) `sns.gridplot`; C) `sns.scattergrid`; D) `sns.matrixplot`
Answer: A
Explanation: Plots all variable pairs.
5. Categorical count plot:
A) `sns.countplot`; B) `sns.catplot(kind="count")`; C) both; D) none
Answer: C
Explanation: countplot and catplot(kind="count") equivalent.

6. Theme setting:
A) `sns.set_theme()`; B) `sns.theme()`; C) `sns.style()`; D) `sns.set_style()` only
Answer: A
Explanation: `set_theme` controls style/palette/context.
7. Regression plot:
A) `sns.regplot`; B) `sns.regr`; C) `sns.lmplot`; D) both A and C
Answer: D
Explanation: `regplot` (axes-level) and `lmplot` (figure-level).
8. Color palette:
A) `sns.color`; B) `sns.set_palette`; C) `sns.palette_set`; D) `sns.colorset`
Answer: B
Explanation: Sets default color cycle.
9. Boxplot:
A) `sns.box`; B) `sns.boxplot`; C) `sns.violin`; D) `sns.swarm`
Answer: B
Explanation: Displays distributions via quartiles/outliers.
10. Save figure (Matplotlib):
A) `plt.save(...)`; B) `plt.savefig(...)`; C) `sns.savefig(...)`; D) `sns.save(...)`
Answer: B
Explanation: Seaborn uses Matplotlib under the hood.
-

14) Machine Learning: Introduction & Types (10)

1. Supervised learning uses:
A) labeled data; B) unlabeled data; C) reinforcement rewards; D) no data
Answer: A
Explanation: Targets (labels) guide learning.
2. Unsupervised learning includes:
A) regression; B) clustering; C) classification; D) boosting
Answer: B
Explanation: Finds patterns without labels.
3. Reinforcement learning core element:
A) optimizer; B) reward signal; C) labels; D) embeddings
Answer: B
Explanation: Agent learns via rewards and penalties.
4. Overfitting means:
A) high bias; B) high variance; C) undertraining; D) balanced
Answer: B
Explanation: Model fits noise; poor generalization.

5. Train/test split purpose:
A) faster code; B) generalization estimate; C) more features; D) regularization
Answer: B
Explanation: Evaluate performance on unseen data.
6. Cross-validation helps:
A) reduce data; B) robust evaluation; C) increase features; D) data leakage
Answer: B
Explanation: Averages performance over folds.
7. Feature scaling important for:
A) trees; B) distance-based models; C) rule-based; D) text only
Answer: B
Explanation: KNN/SVM/logistic benefit from normalized scales.
8. Dimensionality reduction example:
A) PCA; B) SVM; C) KNN; D) Naive Bayes
Answer: A
Explanation: PCA projects features to lower dimensions.
9. Bias-variance tradeoff balances:
A) training time; B) model complexity; C) memory; D) GPU usage
Answer: B
Explanation: Simpler models reduce variance; complex reduce bias.
10. Data leakage occurs when:
A) test info in train; B) train info in test only; C) scaling after split; D) no split
Answer: A
Explanation: Using test data during training inflates metrics.
-

15) Scikit-Learn (10)

1. Train/test split function:
A) train_test_split; B) split_train_test; C) test_train_split; D) split
Answer: A
Explanation: In sklearn.model_selection.
2. Pipeline class:
A) sklearn.pipe; B) sklearn.pipeline.Pipeline; C) sklearn.Pipeline; D) pipeline.Pipe
Answer: B
Explanation: Chains transformers and estimator.
3. Standardization scaler:
A) MinMaxScaler; B) StandardScaler; C) RobustScaler; D) Normalizer
Answer: B
Explanation: Zero mean, unit variance.

4. Fit model method:
A) fit(); B) train(); C) run(); D) learn()
Answer: A
Explanation: fit learns parameters from data.
 5. Predict method returns:
A) probabilities only; B) labels/values; C) scores only; D) raw inputs
Answer: B
Explanation: predict returns classes or numeric outputs.
 6. Grid search class:
A) GridSearch; B) GridCV; C) GridSearchCV; D) SearchGrid
Answer: C
Explanation: Performs exhaustive hyperparameter search.
 7. Randomized search class:
A) RandomSearchCV; B) RandomizedSearchCV; C) RandomCV; D) RandGrid
Answer: B
Explanation: Samples hyperparameters randomly.
 8. k-NN classifier:
A) KNeighborsClassifier; B) KNNClassifier; C) NearestClassifier; D) KNeighbors
Answer: A
Explanation: In sklearn.neighbors.
 9. Linear regression class:
A) LinearRegression; B) LinearModel; C) OLS; D) LR
Answer: A
Explanation: Basic least squares model.
 10. OneHot encoder:
A) OneHotEncoder; B) HotEncoder; C) CategoricalEncoder; D) CategoryHot
Answer: A
Explanation: Converts categorical to binary columns.
-

16) Regression (10)

1. MSE stands for:
A) Mean Square Error; B) Mean Sum Error; C) Maximum Square Error; D) Mean Std Error
Answer: A
Explanation: Average of squared residuals.
2. R^2 measures:
A) residual variance; B) explained variance proportion; C) accuracy; D) precision
Answer: B
Explanation: Fraction of variance explained by the model.

3. Linear regression assumption includes:
A) homoscedasticity; B) non-linearity; C) discontinuity; D) no noise
Answer: A
Explanation: Constant error variance expected.
4. Regularization reduces:
A) bias; B) variance; C) data; D) labels
Answer: B
Explanation: Penalizes complexity to reduce overfitting.
5. Ridge penalty is:
A) L1; B) L2; C) Elastic; D) None
Answer: B
Explanation: L2 shrinks coefficients without setting to zero.
6. Lasso penalty:
A) L2; B) L1; C) L0; D) Huber
Answer: B
Explanation: L1 encourages sparsity (zero coefficients).
7. ElasticNet combines:
A) L1+L2; B) L2+L3; C) L1+L0; D) None
Answer: A
Explanation: Mix of ridge and lasso penalties.
8. MAE is robust to:
A) outliers; B) collinearity; C) scaling; D) missing values
Answer: A
Explanation: Linear penalty less sensitive to extremes than MSE.
9. Polynomial features add:
A) interactions/nonlinear terms; B) regularization; C) normalization; D) labels
Answer: A
Explanation: Extends feature space to capture curvature.
10. Residual plot ideally shows:
A) pattern; B) randomness; C) trend; D) slope
Answer: B
Explanation: No structure indicates good fit assumptions.
-

17) Classification (10)

1. Logistic regression outputs:
A) labels only; B) probabilities via sigmoid; C) distances; D) clusters
Answer: B
Explanation: Sigmoid maps logits to $[0,1]$.

2. Decision boundary is:
A) threshold; B) surface separating classes; C) dataset; D) loss
Answer: B
Explanation: Where predicted class changes.
3. SVM kernel maps:
A) features to higher dimension; B) labels to numbers; C) decrease dims; D) loss to 0
Answer: A
Explanation: Enables linear separation in transformed space.
4. k in k-NN chooses:
A) neighbors count; B) features; C) labels; D) clusters
Answer: A
Explanation: Number of nearest neighbors to vote/regress.
5. Naive Bayes assumes:
A) feature independence; B) feature dependence; C) linearity; D) nonparametric
Answer: A
Explanation: Conditional independence simplifies probabilities.
6. ROC curve plots:
A) TPR vs FPR; B) Precision vs Recall; C) Accuracy vs Error; D) Loss vs Epoch
Answer: A
Explanation: Across thresholds, trade-off of sensitivity vs fall-out.
7. AUC near 0.5 implies:
A) perfect; B) random; C) strong; D) overfit
Answer: B
Explanation: AUC=0.5 equals random guessing.
8. Class imbalance handled via:
A) resampling; B) class weights; C) appropriate metrics; D) all
Answer: D
Explanation: Combine techniques for robust performance.
9. Softmax outputs:
A) binary prob; B) multi-class distribution; C) logits; D) loss
Answer: B
Explanation: Normalizes to probabilities that sum to 1.
10. Decision tree splits by:
A) Gini/Entropy; B) MAE; C) MSE only; D) KL only
Answer: A
Explanation: Measures impurity for classification.
-

18) Ensemble Learning (10)

1. Bagging reduces:
A) bias; B) variance; C) features; D) labels
Answer: B
Explanation: Averaging across bootstrapped models stabilizes predictions.
2. Boosting reduces:
A) variance; B) bias; C) both; D) neither
Answer: B
Explanation: Sequentially focuses on errors to increase fit.
3. Random Forest uses:
A) single tree; B) many trees with feature subsampling; C) SVM; D) KMeans
Answer: B
Explanation: Decorrelates trees via random features per split.
4. AdaBoost focuses on:
A) easy examples; B) hard examples; C) random; D) clusters
Answer: B
Explanation: Increases weights on misclassified points.
5. Gradient Boosting optimizes:
A) loss via gradients; B) accuracy directly; C) ROC only; D) distance
Answer: A
Explanation: Fits new learners to negative gradients of loss.
6. Stacking combines:
A) same model; B) base models + meta-model; C) features only; D) datasets
Answer: B
Explanation: Meta-learner uses base predictions as inputs.
7. XGBoost advantages include:
A) speed & regularization; B) only speed; C) only accuracy; D) unsupervised
Answer: A
Explanation: Efficient, supports L1/L2 and advanced features.
8. Feature importance in trees via:
A) impurity decrease; B) PCA; C) weights; D) counts
Answer: A
Explanation: Measures average reduction in impurity.
9. ExtraTrees add randomness by:
A) random thresholds; B) random labels; C) random epochs; D) random loss
Answer: A
Explanation: Random split thresholds lower variance further.
10. Overfitting in boosting mitigated by:
A) early stopping; B) increasing depth; C) removing validation; D) no regularization

Answer: A

Explanation: Stop when validation score stalls.

19) Natural Language Processing (10)

1. Tokenization splits:

A) corpus into docs; B) text into tokens; C) labels; D) vectors

Answer: B

Explanation: Tokens are words/subwords/characters.

2. Stopwords are:

A) informative words; B) common function words; C) unique words; D) entities

Answer: B

Explanation: Often removed to reduce noise.

3. Stemming vs Lemmatization:

A) same; B) lemma uses vocabulary; C) stem uses grammar; D) neither

Answer: B

Explanation: Lemmas are dictionary forms; stems are crude truncations.

4. Bag-of-Words ignores:

A) word order; B) frequencies; C) tokens; D) vocabulary

Answer: A

Explanation: BOW counts words regardless of order.

5. TF-IDF downweights:

A) rare; B) frequent; C) short; D) long words

Answer: B

Explanation: Penalizes common terms to highlight informative ones.

6. n-grams capture:

A) global topics; B) local word sequences; C) embeddings; D) labels

Answer: B

Explanation: Preserve short-range order information.

7. Named Entity Recognition extracts:

A) POS; B) entities; C) syntax trees; D) lemmas

Answer: B

Explanation: Finds people, locations, orgs, etc.

8. POS tagging finds:

A) entities; B) grammatical roles; C) topics; D) sentiments

Answer: B

Explanation: Assigns tags like NN/VB/JJ.

9. Sentiment analysis often uses:

A) regression; B) classification; C) clustering; D) reinforcement

Answer: B

Explanation: Categorizes polarity (e.g., positive/negative).

10. Word normalization includes:

A) lowercasing; B) punctuation stripping; C) stemming; D) all

Answer: D

Explanation: Standard preprocessing pipeline.

20) NLTK (10)

1. NLTK stands for:

A) Natural Language Toolkit; B) Natural Language Kit; C) NLP Toolkit; D) Neural Language Toolkit

Answer: A

Explanation: A classic Python NLP library.

2. Tokenizer example:

A) nltk.tokenize.word_tokenize; B) nltk.word; C) nltk.tokens; D) nltk.split

Answer: A

Explanation: Word-level tokenizer using Punkt.

3. Stopwords via:

A) nltk.corpus.stopwords; B) nltk.stop; C) nltk.common; D) nltk.words

Answer: A

Explanation: Contains multilingual stopword lists.

4. POS tagger function:

A) nltk.pos_tag; B) nltk.tag_pos; C) nltk.tag; D) nltk.pos

Answer: A

Explanation: Applies tagger to token list.

5. NER in NLTK uses:

A) ne_chunk; B) ner; C) chunk; D) entity_tag

Answer: A

Explanation: Chunker based on a trained classifier.

6. Stemming classes include:

A) PorterStemmer; B) SnowballStemmer; C) both; D) None

Answer: C

Explanation: NLTK implements several stemmers.

7. Lemmatizer:

A) WordNetLemmatizer; B) Lemma; C) NERLemma; D) WordLemma

Answer: A

Explanation: Requires WordNet corpus.

8. Download corpora:

A) nltk.download('all'); B) nltk.get; C) nltk.install; D) nltk.corpus.download

Answer: A

Explanation: Downloader fetches required resources.

9. Sentence tokenizer:

A) sent_tokenize; B) sentence_token; C) token_sent; D) sent_split

Answer: A

Explanation: Splits text into sentences.

10. NLTK data directory configurable via:

A) nltk.setdir; B) NLTK_DATA; C) nltk.data.path; D) nltk.path

Answer: C

Explanation: Add paths to list for resource discovery.

21) Evaluation Metrics (10)

1. Accuracy =

A) $TP/(TP+FP)$; B) $(TP+TN)/Total$; C) $TP/(TP+FN)$; D) $TN/(TN+FP)$

Answer: B

Explanation: Correct predictions over all predictions.

2. Precision =

A) $TP/(TP+FP)$; B) $TP/(TP+FN)$; C) $TN/(TN+FP)$; D) $(TP+TN)/Total$

Answer: A

Explanation: Among predicted positives, proportion that are true.

3. Recall =

A) $TP/(TP+FP)$; B) $TP/(TP+FN)$; C) $TN/(TN+FP)$; D) $FP/(FP+TN)$

Answer: B

Explanation: Among actual positives, proportion correctly found.

4. F1 Score =

A) harmonic mean of precision & recall; B) arithmetic mean; C) geometric mean; D) max

Answer: A

Explanation: Balances precision/recall.

5. Confusion matrix shows:

A) losses; B) $TP/TN/FP/FN$; C) ROC; D) PR

Answer: B

Explanation: Counts of outcomes across true/predicted classes.

6. MAE is for:

A) classification; B) regression; C) clustering; D) RL

Answer: B

Explanation: Average of absolute errors.

7. AUC-PR preferred when:

A) balanced classes; B) imbalanced classes; C) multi-label always; D) regression

Answer: B

Explanation: Sensitive to class imbalance.

8. Log loss penalizes:

A) wrong confident predictions; B) correct; C) neutral only; D) outliers

Answer: A

Explanation: High penalty for confident but incorrect probabilities.

9. MAPE sensitive to:

A) zeros in targets; B) class imbalance; C) features; D) variance

Answer: A

Explanation: Division by zero or near-zero inflates error.

10. Matthews Correlation Coefficient is:

A) binary-only metric robust to imbalance; B) regression; C) clustering; D) time series

Answer: A

Explanation: Balanced measure using all confusion matrix entries.

22) MLP Feed Forward Neural Network (10)

1. Common hidden activation:

A) ReLU; B) Softmax; C) Identity; D) None

Answer: A

Explanation: ReLU mitigates vanishing gradients and is efficient.

2. Output activation for binary classification:

A) Softmax; B) Sigmoid; C) ReLU; D) Tanh

Answer: B

Explanation: Sigmoid outputs single probability.

3. Backpropagation updates:

A) inputs; B) weights via gradients; C) labels; D) loss

Answer: B

Explanation: Gradient descent adjusts weights.

4. Vanishing gradients occur with:

A) ReLU; B) sigmoid/tanh deep nets; C) linear; D) none

Answer: B

Explanation: Saturating activations produce small gradients.

5. Dropout helps:

A) overfitting; B) underfitting; C) data leakage; D) scaling

Answer: A

Explanation: Randomly omits units to improve generalization.

6. Batch normalization:

A) normalizes activations; B) normalizes labels; C) decreases features; D) increases epochs

Answer: A

Explanation: Stabilizes training by standardizing layer inputs.

7. Optimizer example:

A) Adam; B) SGD; C) RMSProp; D) all

Answer: D

Explanation: Multiple optimizers are commonly used.

8. Loss for regression:

A) Cross-entropy; B) MSE; C) Hinge; D) Logistic

Answer: B

Explanation: MSE fits continuous targets.

9. Weight initialization helps:

A) convergence; B) labels; C) loss definition; D) metrics

Answer: A

Explanation: Good initialization speeds and stabilizes training.

10. Early stopping monitors:

A) training loss only; B) validation metric; C) test; D) features

Answer: B

Explanation: Stop when validation performance stops improving.

23) Convolutional Neural Network (CNN) (10)

1. Convolution extracts:

A) global stats; B) local features; C) labels; D) embeddings only

Answer: B

Explanation: Kernels detect local patterns like edges/textures.

2. Pooling reduces:

A) dimensions; B) labels; C) weights; D) epochs

Answer: A

Explanation: Downsampling increases invariance and reduces compute.

3. Common kernel sizes include:

A) 1×1, 3×3, 5×5; B) 2×2 only; C) 10×10 only; D) none

Answer: A

Explanation: Small kernels are efficient and expressive.

4. Padding preserves:

A) output size; B) labels; C) learning rate; D) loss

Answer: A

Explanation: Zero-padding keeps spatial dimensions.

5. Stride controls:

A) step of kernel; B) number of layers; C) classes; D) channels

Answer: A

Explanation: Larger stride reduces output size.

6. Batch norm in CNN applied:

A) before/after conv; B) only at end; C) only inputs; D) never

Answer: A

Explanation: Often after conv and before activation.

7. Transfer learning uses:

A) random init; B) pre-trained weights; C) no training; D) labels only

Answer: B

Explanation: Start from features learned on large datasets.

8. Data augmentation helps:

A) generalization; B) decrease data; C) overfit; D) labeling

Answer: A

Explanation: Synthetic variations expand training diversity.

9. Global average pooling replaces:

A) fully connected layers; B) convolutions; C) batch norm; D) dropout

Answer: A

Explanation: Summarizes feature maps into one value per channel.

10. Dilated convolution:

A) reduces field; B) expands receptive field; C) removes padding; D) increases parameters

Answer: B

Explanation: Inserts gaps to capture larger context.

24) Recurrent Neural Networks (RNNs) (10)

1. RNNs are good for:

A) images; B) sequences/time series; C) tabular only; D) clustering

Answer: B

Explanation: Maintain state across time steps.

2. Hidden state updates:

A) independent; B) depends on previous state; C) random; D) static

Answer: B

Explanation: Each step uses previous state to propagate information.

3. Vanishing gradient is common in:

A) deep RNNs; B) CNNs only; C) MLP only; D) none

Answer: A

Explanation: Long dependencies cause small gradients.

4. Teacher forcing applies to:

A) seq2seq training; B) inference only; C) regression; D) clustering

Answer: A

Explanation: Feed ground-truth outputs during training.

5. Bidirectional RNN reads:

A) forward only; B) both directions; C) random; D) interleaved only

Answer: B

Explanation: Combines past and future context.

6. Sequence padding needed for:

A) fixed batch sizes; B) variable lengths; C) embeddings; D) labels

Answer: B

Explanation: Aligns samples to same length for batch processing.

7. Truncated BPTT limits:

A) memory via shorter unroll; B) labels; C) features; D) epochs

Answer: A

Explanation: Cuts backprop through long sequences.

8. RNN cell types include:

A) Vanilla, LSTM, GRU; B) CNN; C) MLP; D) SVM

Answer: A

Explanation: Variants to manage sequence dependencies.

9. Gradient clipping helps:

A) exploding gradients; B) vanishing gradients; C) loss computation; D) data loading

Answer: A

Explanation: Caps magnitude to stabilize training.

10. Packed sequences (frameworks) are used to:

A) sort labels; B) handle variable-length efficiently; C) normalize tokens; D) increase epochs

Answer: B

Explanation: Skip computation on pad tokens.

25) Long Short-Term Memory (LSTM) (10)

1. LSTM addresses:

A) overfitting; B) vanishing gradients; C) scaling; D) label noise

Answer: B

Explanation: Gate mechanisms preserve gradients across time.

2. LSTM has:

A) input/forget/output gates; B) no gates; C) convolution; D) pooling

Answer: A

Explanation: Controls information flow.

3. Cell state in LSTM is:

A) short-term only; B) long-term memory; C) label; D) gradient

Answer: B

Explanation: Carries info with minimal modification.

4. Forget gate controls:

A) updates; B) what to discard; C) outputs; D) inputs

Answer: B

Explanation: Scales previous cell state via sigmoid.

5. Peephole connections allow:

A) cell state to gates; B) labels to gates; C) outputs to inputs; D) none

Answer: A

Explanation: Gates can “peek” at cell state.

6. LSTM vs GRU difference:

A) GRU has fewer gates; B) GRU has more gates; C) LSTM no gates; D) both same

Answer: A

Explanation: GRU merges gates, reducing parameters.

7. LSTM common activations:

A) tanh + sigmoid gates; B) ReLU only; C) softmax; D) linear

Answer: A

Explanation: Gates use sigmoid; cell update uses tanh.

8. LSTM applied to:

A) sequences (text, time series); B) images; C) graphs; D) trees

Answer: A

Explanation: Designed for temporal/ordered data.

9. Coupled input-forget gate variant:

A) ties gates together; B) removes output; C) adds extra state; D) none

Answer: A

Explanation: Reduces parameters, sometimes improves performance.

10. LSTM regularization commonly includes:

A) recurrent dropout; B) weight decay only; C) batch norm only; D) none

Answer: A

Explanation: Dropout on recurrent connections can help.

26) Gated Recurrent Unit (GRU) (10)

1. GRU gates:

A) update & reset; B) input & output; C) forget & input; D) none

Answer: A

Explanation: Two gates control memory flow.

2. GRU has:

A) separate cell state; B) combined hidden state; C) no hidden state; D) only output

Answer: B

Explanation: Merges cell and hidden states.

3. GRU compared to LSTM:

A) more parameters; B) fewer parameters; C) same parameters; D) no gates

Answer: B

Explanation: Simpler structure with fewer gates.

4. Reset gate controls:

A) new info; B) forgetting previous info; C) output; D) labels

Answer: B

Explanation: Determines how much past info to drop.

5. Update gate balances:

A) old vs new info; B) outputs; C) inputs; D) gradients

Answer: A

Explanation: Interpolates between previous and candidate state.

6. GRU often performs:

A) similar to LSTM; B) always worse; C) always better; D) same speed

Answer: A

Explanation: Comparable accuracy with faster training sometimes.

7. GRU gradient issues are:

A) no vanishing; B) reduced vanishing vs vanilla RNN; C) more vanishing than LSTM;
D) unrelated

Answer: B

Explanation: Gating helps maintain gradients.

8. GRU activation functions:

A) sigmoid/tanh; B) ReLU only; C) softmax; D) identity

Answer: A

Explanation: Sigmoid gates, tanh for candidate.

9. GRU use-cases include:

A) time series forecasting; B) image classification only; C) clustering; D)
reinforcement only

Answer: A

Explanation: Sequential modeling is typical.

10. GRU parameter efficiency helps:

A) small datasets; B) large datasets only; C) no impact; D) static graphs

Answer: A

Explanation: Fewer parameters can reduce overfitting.

27) Word Embeddings (10)

1. Word2Vec learns via:
A) CBOW/Skip-gram; B) SVM; C) PCA; D) KMeans
Answer: A
Explanation: Predicts target from context or vice versa.
2. GloVe is:
A) global co-occurrence model; B) convolution; C) RNN; D) transformer
Answer: A
Explanation: Factorizes co-occurrence statistics.
3. Embedding dimension controls:
A) label count; B) vector size; C) epochs; D) batch size
Answer: B
Explanation: Higher dims capture more semantic detail.
4. OOV words are:
A) in vocab; B) out-of-vocabulary; C) stopwords; D) entities
Answer: B
Explanation: Not present in training dictionary.
5. Subword embeddings used by:
A) FastText; B) Word2Vec; C) GloVe; D) SVM
Answer: A
Explanation: Builds word vectors from character n-grams.
6. Contextual embeddings example:
A) BERT; B) Word2Vec; C) GloVe; D) PCA
Answer: A
Explanation: Vectors depend on sentence context.
7. Cosine similarity used to:
A) compare embedding vectors; B) scale features; C) cluster labels; D) reduce dims
Answer: A
Explanation: Measures angle between vectors.
8. Embedding layer in NN maps:
A) tokens → dense vectors; B) labels → tokens; C) features → labels; D) loss → accuracy
Answer: A
Explanation: Lookup table trained end-to-end.
9. Negative sampling in Word2Vec:
A) speeds training; B) slows it; C) overfits; D) is unrelated
Answer: A
Explanation: Approximates softmax with few negative examples.
10. Sentence embeddings can be built via:
A) averaging word vectors; B) transformer pooling; C) both; D) none

Answer: C

Explanation: Multiple strategies exist.

28) Selection of Microsoft Azure AI Service (10)

1. For custom LLM chat, choose:

A) Azure AI Vision; B) Azure OpenAI Service; C) Azure AI Speech; D) Azure AI Language

Answer: B

Explanation: Provides GPT-family models and APIs.

2. OCR & document extraction:

A) Azure AI Document Intelligence; B) Azure AI Language; C) Azure AI Search; D) Azure OpenAI

Answer: A

Explanation: Formerly Form Recognizer for forms/invoices/IDs.

3. Semantic search over enterprise data:

A) Azure AI Search; B) Azure SQL; C) Azure AI Speech; D) Azure ML

Answer: A

Explanation: Indexes content and provides semantic ranking.

4. Sentiment & key phrase extraction (no training):

A) Azure AI Language; B) Azure ML; C) Azure Databricks; D) Azure Vision

Answer: A

Explanation: Prebuilt language APIs.

5. Pretrained image classification & tagging:

A) Azure AI Vision; B) Azure ML; C) Azure Kinect; D) Azure Batch

Answer: A

Explanation: Computer Vision APIs for images.

6. Speech-to-text and text-to-speech:

A) Azure AI Speech; B) Azure OpenAI; C) Azure ML; D) Azure Functions

Answer: A

Explanation: Unified Speech SDK covers STT/TTS.

7. Custom ML modeling & deployment:

A) Azure Machine Learning; B) Azure AI Language; C) Azure OpenAI; D) Power BI

Answer: A

Explanation: ML platform for training/serving.

8. Content moderation and filters with LLMs:

A) Azure Content Safety; B) Azure AI Vision; C) Azure Key Vault; D) Azure Monitor

Answer: A

Explanation: Safety filters for text/images.

9. Real-time translation:
A) Azure AI Speech (Translation); B) Azure AI Language; C) Azure ML; D) Azure Batch
Answer: A
Explanation: Speech translation supports multi-language streaming.
10. Conversational Q&A over PDFs/websites:
A) Azure AI Search + Azure OpenAI; B) Azure ML only; C) Azure AI Vision only; D) Key Vault
Answer: A
Explanation: Index with Search, ground LLM responses on content.
-

29) Deploy Microsoft Azure AI Services (10)

1. Authentication typically via:
A) API keys or Managed Identities; B) passwords; C) cookies; D) anonymous
Answer: A
Explanation: Use key-based auth or AAD MSI for secure access.
2. Private network access for endpoints via:
A) Public IP; B) Private Endpoints; C) DNS only; D) RBAC only
Answer: B
Explanation: Private endpoints map services to VNet addresses.
3. Deploy Azure ML online endpoint type:
A) batch; B) real-time; C) offline; D) async only
Answer: B
Explanation: Online endpoints serve low-latency inference.
4. Model registry in Azure ML stores:
A) datasets; B) models & versions; C) metrics only; D) pipelines
Answer: B
Explanation: Central hub for models with lineage.
5. Autoscale compute for inference via:
A) Managed Online Endpoints or AKS; B) VM only; C) Functions only; D) App Service only
Answer: A
Explanation: Managed endpoints auto-scale; AKS supports high scale.
6. Key Vault used to:
A) store secrets/keys; B) store datasets; C) store models; D) store logs
Answer: A
Explanation: Secure secret management for endpoints.
7. Monitoring inference with:
A) Azure Monitor + Application Insights; B) Log Analytics only; C) none; D) DevOps only

Answer: A

Explanation: Collect metrics, traces, logs and alerts.

8. Azure OpenAI quotas/limits managed by:

A) subscription quota; B) Key Vault; C) ACR; D) Data Factory

Answer: A

Explanation: Capacity quotas per region/model.

9. Blue/green deployment in Azure ML:

A) split traffic between revisions; B) retrain model; C) resize compute; D) rotate keys

Answer: A

Explanation: Route percentage of traffic to new revision for safe rollout.

10. Data encryption at rest in Azure:

A) disabled; B) platform-managed keys enabled by default; C) only customer-managed; D) none

Answer: B

Explanation: Storage encrypted; CMK optional for additional control.

30) Development & Implementation of Different Microsoft Azure AI Services (10)

1. SDK for Azure AI Language (Text Analytics) in Python:

A) azure-ai-textanalytics; B) azure-ai-language; C) azure-language; D) azure-text

Answer: A

Explanation: Provides sentiment, key phrase, PII detection APIs.

2. SDK for Azure AI Speech in Python:

A) azure-ai-speech; B) azure-cognitiveservices-speech; C) azure-speech; D) speech-sdk-python

Answer: B

Explanation: Official Speech SDK package name.

3. SDK for Azure AI Document Intelligence in Python:

A) azure-ai-formrecognizer; B) azure-doc-intel; C) azure-document; D) azure-ai-doc

Answer: A

Explanation: Analyze receipts, invoices, IDs.

4. SDK for Azure AI Vision (Computer Vision) in Python:

A) azure-ai-vision; B) azure-cognitiveservices-vision-computervision; C) azure-vision; D) cv-azure

Answer: B

Explanation: Computer Vision client for tagging/OCR.

5. Azure ML Python SDK core package:

A) azure-ml; B) azure-ai-ml; C) azureml-core only; D) azure-machine-learning

Answer: B

Explanation: v2 SDK for workspaces, jobs, endpoints.

6. Indexing external data sources for Azure AI Search uses:

A) indexers & skillsets; B) labels; C) functions; D) AKS

Answer: A

Explanation: Indexers pull data; skillsets enrich content.

7. Grounding LLM answers on enterprise content:

A) Azure AI Search + Azure OpenAI; B) Azure ML alone; C) Key Vault; D) Power Automate

Answer: A

Explanation: Retrieve relevant docs then generate answers.

8. Conversation safety controls for LLM apps:

A) Azure Content Safety; B) Azure Monitor; C) AKS; D) ACR

Answer: A

Explanation: Detects harmful content in prompts/outputs.

9. Deploy Python app with Azure OpenAI securely:

A) Use Managed Identity & private endpoints; B) public key in code; C) hardcode passwords; D) expose endpoint publicly

Answer: A

Explanation: Follows least privilege and network isolation.

10. CI/CD for ML endpoints commonly via:

A) GitHub Actions/Azure DevOps; B) local scripts only; C) email; D) manual portal clicks

Answer: A

Explanation: Automates build, test, deploy across environments.